



CEGT: Smart contract vulnerability detection via Connectivity-Enhanced GCN-Transformer^{☆,☆☆}

Jiandong Shang^{a,b}, Jiaru Li^a, Yizhe Sui^a, Hengliang Guo^{a,b}, Xu Gao^a, Dujuan Zhang^b, Yang Guo^b, Gang Wu^{a,b,*,*}

^a School of Computer and Artificial Intelligence, University Zhengzhou, No.100, Kexue Avenue, Zhengzhou, 450001, China

^b National Supercomputing Center in Zhengzhou, University, Zhengzhou, Southeast corner of Changchun Road and Fengyang Street intersection, Zhengzhou, 450001, China

ARTICLE INFO

Keywords:

Smart contract
Vulnerability detection
Neural networks

ABSTRACT

The deployment of smart contracts on blockchains is rising rapidly. Accurate detection of security vulnerabilities in smart contracts can significantly minimize property losses. However, most existing machine learning (ML)-based models for smart contract vulnerability detection overlook the contract graph structures and sequence information, reducing detection effectiveness. This study presents a Connectivity-Enhanced GCN-Transformer (CEGT), a method for detecting smart contract vulnerability detection that integrates graph and sequence models to enhance vulnerability detection accuracy. We improve node connectivity by identifying additional paths between nodes in the graph and augment the representation capability of node features through an additional orthogonal transformation layer, which performs an orthogonal transformation on the weight matrix. Moreover, we designed a novel attention mechanism, termed the dynamic attention mechanism, based on the sequence model and inspired by the concept of dynamic routing in capsule networks. Such a dynamic attention mechanism within the sequence model is introduced to integrate structural and sequential information of smart contracts, thereby enhancing vulnerability in detection accuracy. Our experiments demonstrate that CEGT surpasses state-of-the-art methods in detecting Reentrancy, Timestamp dependence, and Integer overflow vulnerabilities, achieving F1 scores of 93.47%, 89.33%, and 91.27%, respectively. This enables us to achieve greater accuracy in detecting smart contract vulnerabilities, helping to identify potential risks, reduce security threats, and ensure the reliability and safety of blockchain applications.

Editor's note: Open Science material was validated by the Journal of Systems and Software Open Science Board.

1. Introduction

Blockchain is a decentralized distributed ledger technology in which all miners in the network adhere to a consensus mechanism for collective ledger maintenance. Recently, blockchain has advanced swiftly and gained extensive application across multiple fields, including finance, supply chain management, and healthcare (Casino et al., 2019). Smart contracts are a key application of the blockchain, functioning as programs that execute automatically on the blockchain platform (Liu et al., 2024a). Once the trigger conditions are met, smart contracts autonomously perform the corresponding operations, thereby effectively reducing transaction costs and enhancing efficiency.

As the deployment of smart contracts on the blockchain increases (Wu et al., 2024), and given the immutability of blockchain with over 15,000 new contracts added to the Ethereum platform in the past six months (Liu et al., 2021b), security issues related to smart contracts have become a critical concern. Historically, significant losses have occurred due to smart contract security vulnerabilities. For instance, the 2016 Decentralized Autonomous Organization (DAO) incident involved attackers exploiting a recursive call vulnerability in smart contracts, resulting in approximately \$60 million in Ether losses (Mehar et al., 2019).

[☆] The research was supported by Science and Technology Innovation 2023 - 'Next-Generation Artificial Intelligence' Major Project (Project No. 2023ZD0120604) and the Major Science and Technology Project in Henan Province (Project No. 221100210400).

^{☆☆} Editor: Heiko Koziolok.

* Corresponding author.

E-mail addresses: sjd@zzu.edu.cn (J. Shang), ruruli@gs.zzu.edu.cn (J. Li), a1522446272@gs.zzu.edu.cn (Y. Sui), guohengliang@zzu.edu.cn (H. Guo), gaoxu@zzu.edu.cn (X. Gao), duduzdi@zzu.edu.cn (D. Zhang), guoyang@zzu.edu.cn (Y. Guo), gangwu@zzu.edu.cn (G. Wu).

<https://doi.org/10.1016/j.jss.2025.112454>

Received 14 November 2024; Received in revised form 28 March 2025; Accepted 7 April 2025

Available online 21 April 2025

0164-1212/© 2025 Elsevier Inc. All rights reserved, including those for text and data mining, AI training, and similar technologies.

Currently, traditional vulnerability detection tools for smart contracts, including symbol execution, fuzzy testing, formal verification, and static analysis methods, have been extensively developed. However, conventional detection methods still face several issues: (1) They rely heavily on rigid rules and expert experience, making it easier for attackers to exploit existing vulnerabilities, leading to potential losses; (2) The update speed of expert rules often fails to keep pace with the rapid development of smart contracts. Consequently, many researchers are exploring the application of machine learning (ML) for vulnerability detection in smart contracts.

Some researchers have explored the detection of smart contract vulnerabilities through traditional ML techniques. Liu et al. (2018) introduced s-gram, which employs static analysis to extract semantic information and generate token sequences from the abstract syntax tree (AST) node types. The automatic feature extraction capabilities of deep learning (DL) and neural networks have led some researchers to adopt DL-based approaches for vulnerability detection. Subsequently, some researchers have tried to identify the structural relationships within smart contract codes to improve vulnerability detection effectiveness. Some of them have focused on transforming contract codes into contract graphs. DR-GCN (Zhuang et al., 2020) was the first to utilize Graph Convolutional Networks (GCN) for transforming smart contracts into contract graphs to develop detection models. Some studies have suggested visualizing contract code. For instance, the SC-VDM (Zhou et al., 2021) model converts the smart contract bytecode into a gray matrix picture. Some studies utilize multi-feature fusion to detect vulnerability detection by extracting features from various aspects of smart contracts (Wang et al., 2024).

However, the existing learning-based methods for smart contract vulnerability detection face several challenges: (1) The sequence models that convert smart contracts into sequence data focus solely on the contextual relationships within the text sequences, neglecting the semantic relationships (Liu et al., 2024b), which results in missing key information. (2) GCN considers only the local neighborhood of nodes, disregarding the path information between non-adjacent nodes, which is crucial for vulnerability detection in contract graphs. (3) Although the multi-feature fusion method can comprehensively incorporate both the structural and sequence information from contract graphs, merely splicing feature vectors for vulnerability detection may lead to issues such as information redundancy and expanded feature dimension (Liu et al., 2023).

In this study, CEGT overcomes the limitations of traditional smart contract vulnerability detection through several innovations. First, it enhances node connectivity. This improvement boosts the representation capability of node features, leading to higher detection accuracy. Second, CEGT incorporates an improved sequence model. This model effectively captures sequential information in contracts, which strengthens the detection of complex logical vulnerabilities. Finally, CEGT avoids simple feature concatenation. This approach reduces feature dimensionality. These improvements enable CEGT to achieve higher precision in smart contract vulnerability detection.

The main contributions of this study are as follows:

- (1) We integrate sequence and graph models for smart contract vulnerability detection, taking into account both the sequence and structural information of the smart contract.
- (2) We enhance node connectivity in the graph model and employ orthogonal transformations to strengthen the representation capability of the node features, thereby improving the detection accuracy of the model.
- (3) We introduce a dynamic routing attention mechanism that iteratively adjusts the weights through multiple iterations to enhance feature extraction among inputs across multiple levels.
- (4) Our model demonstrates its superiority through comparisons with state-of-the-art vulnerability detection models.

The remainder of this paper is organized as follows. Section 2 reviews foundational approaches for detecting smart contract vulnerabilities, covering conventional and learning-based smart contract vulnerability detection methods, and their current limitations. Section 3 outlines the three types of smart contract vulnerabilities examined in this study. Section 4 describes the method implementation, including data preprocessing and the model design. Section 5 evaluates the model detection performance and module improvements through comparison with other models and ablation experiments. Section 6 discusses the experimental results in detail. Section 7 discusses the factors that may affect the detection performance of the model. Finally, Section 8 concludes the study and discusses future research developments and research directions.

2. Related works

2.1. Conventional methods

Some researchers have developed traditional methods for detecting smart contract vulnerabilities. Li et al. (2023a) classified these vulnerabilities of smart contracts into three categories based on their origin: language layer, ethernet virtual machine (EVM) layer, and blockchain system layer, identifying 13 common vulnerability types. They also summarized five main vulnerability detection approaches: symbol execution, fuzzy testing, formal verification, static analysis, and ML. Symbol execution (He et al., 2024) is an analytical technique that analyzes programs by simulating execution with symbolic instead of specific inputs to identify vulnerabilities. Fuzzy testing (Wang et al., 2020) generates test cases with semi-valid data as inputs, monitoring the execution process and results to detect vulnerabilities. Formal validation (You et al., 2012) applies mathematical models and methods to rigorously validate program security utilizing mathematical methods and mathematical language. Static analysis (Liu et al., 2012) examines programs without execution, identifying vulnerabilities by analyzing the code, structure, and documentation of the program. ML methods transform programs into properly structured data, applying a series of processes used as a suitable model improvement. Each method has been extensively researched in recent years. Oyente (Luu et al., 2016), a symbol-execution-based vulnerability detection tool, inputs smart contract bytecode and constructs a control flow graph (CFG). SmartCheck (Tikhomirov et al., 2018), a static analysis tool, detects vulnerabilities by converting the smart contract source code into intermediate representations and matching pre-defined vulnerability patterns.

2.2. ML-based methods

ML has emerged as a promising approach for smart contract vulnerability detection, drawing significant research interest. In 2018, Liu et al. (2018) introduced s-gram, which employs static analysis to extract semantic information, generating token sequences from AST node types. An n-gram language model is then constructed from these tokens to identify vulnerabilities in smart contracts. Liao et al. (2019) introduced SoliAudit, which combines ML with gray box fuzzy testing to detect vulnerabilities independently of expert knowledge or pre-defined patterns. ContractWard (Wang et al., 2021b) utilized bigram features from smart contracts opcodes and applied multiple ML algorithms to detect smart contract vulnerabilities based on the extracted static features.

DL-based methods for smart contract vulnerability detection enable automated extraction of key features, facilitating fast and accurate detection. Some researchers utilize smart contract code as a text sequence, preserving the code's original information flow. Embedding algorithms such as Word2Vec have been applied to transform smart contracts bytecode or source code into vectors, enabling model training

for vulnerability detection. SaferSC (Tann et al., 2018) is the first DL-based smart contract vulnerability detection model to utilize LSTM for smart contract vulnerability detection through opcodes of smart contracts. Tann et al. (2018) trained vectors using LSTM, while Qian et al. (2020) introduced the ReChecker method, significantly enhancing accuracy by converting the smart contract source code into fragments and employing a combination of BiLSTM and attention mechanism to capture the contextual relationships of smart contract, fundamental semantics, and control flow dependencies. Wang et al. (2021a) introduced PSD-OL, a detection method for the Ethereum Ponzi scheme smart contracts. This approach, based on LSTM, comprehensively integrates account and code characteristics of smart contracts and employs an oversampling technique to address the training data imbalance problem, thereby enhancing the accuracy of vulnerability detection. Ashizawa et al. (2022) proposed Eth2Vec, a static analysis tool that leverages neural networks to automatically learn features of vulnerable EVM bytecodes through the neural network, enabling vulnerability detection of smart contract through the similarity comparison with the target EVM bytecodes. Hu et al. (2021) developed an algorithm for slicing smart contract data, while Gupta et al. (2022) classified smart contract security based on the probability values; all of these approaches utilized LSTM for vulnerability detection. The emergence of Transformer and BERT has enabled the treatment of smart contract codes as text sequences. Gong et al. (2023) developed a CFG for smart contract opcodes, circumventing limitations associated with the programming language version. Cao et al. (2023) employed a multilayer Bi-Transformer model architecture for vulnerability detection in smart contracts, while Jeon et al. (2023) applied BERT for processing contracts. Following the rapid advancement of the pre-training technology enhancements to the BERT, several researchers (Wu et al., 2021; Guo et al., 2020) have utilized GraphCodeBERT for vulnerability detection. Wu et al. (2021) introduced a novel method, Peculiar, which leverages a simplified key data flow graph to prevent information overfitting and direct the model's focus toward essential features. Guo et al. (2020) enhanced GraphCodeBERT by incorporating data flow and introducing two pre-training tasks to improve code comprehension.

2.3. GCN-based methods

Incorporating the structural information of smart contracts facilitates a deeper analysis of their logical relationships in the contract. Consequently, beyond treating smart contract code as text to extract for feature extraction, some researchers have enhanced vulnerability detection by identifying semantic relationships within the smart contract code. Notably, several scholars have transformed the contract code into contract graphs for this purpose. Zhuang et al. (2020) introduced the DR-GCN, the first model to convert smart contracts into contract graphs. This approach translates key functions and logical call structures in smart contracts into corresponding nodes and edges, normalizing the contract graphs through standardized operations. Enhanced from the original GCN, the DR-GCN strengthens node connectivity, thereby improving the detection performance of the model. In the TMP method (Zhuang et al., 2020), the smart contract code is converted into a contract graph, introducing a new TMP of the message transmission network to facilitate feature propagation during both the message transmission and readout phases. Building on this approach, Liu et al. proposed the CGE model (Liu et al., 2021b), which integrates the graph neural network with expert knowledge to enhance the accuracy of vulnerability detection. ReVulDL (Zhang et al., 2022) not only detects the Reentrancy vulnerabilities but also locates them through interpretive analysis. Zhen et al. (2024) employed the DA-GNN method to update the node embeddings in the graph attention network using a dual attention mechanism, integrating this information with a self-attention mechanism. In addition, some researchers have proposed converting contract codes into images, thereby framing smart contract vulnerability detection as an image classification. Zhou et al. (2021)

proposed the SC-VDM method, which converts the smart contract bytecode into a gray matrix image, enabling automatic vulnerability detection in smart contracts without expert knowledge. Similarly, Huang (2018) proposed transforming bytecode into RGB color codes and encoding it for model training and learning. Some researchers have employed multi-feature fusion for vulnerability detection. Wang et al. (2024) extracted various features of smart contracts through the stages such as color marking, vocabulary extraction, and character conversion. These features are then integrated into a model fused by different approaches for feature extraction, splicing, and vulnerability detection.

3. Smart contract vulnerability type

This study focuses on the detection of smart contracts according to various security vulnerabilities. We choose to conduct detection experiments on three types of vulnerabilities: Reentrancy vulnerability, Timestamp dependence vulnerability, and Integer overflow vulnerability, because: (1) these vulnerabilities are prevalent and critical, having caused over \$100 million in losses across legal, financial, and other sectors globally (Liu et al., 2024a); and (2) the datasets for these vulnerabilities are well-suited for training deep learning detection models. This section provides a detailed description of the three security vulnerabilities identified in this study.

3.1. Reentrancy vulnerability

Reentrancy is a prevalent and significant vulnerability in smart contracts written in Solidity language, characterized by a loop call flaw. The vulnerability arises primarily with transfer functions and callback functions. In Ethereum, the `call.value()` transfer function is commonly invoked for transaction transfers using smart contracts. Upon invocation of `call.value()` function, the contract receives the transfer and automatically triggers the `fallback()` callback function in the Solidity language mechanism. In addition, the `fallback()` function executes automatically under the following conditions:

- (1) When invoking the contract function, an incorrect function match occurs, resulting in a call error.
- (2) Missing incoming parameters or mismatched parameters occur when invoking the contract function.

As illustrated in Fig. 1, account A automatically invokes the `fallback()` function of contract B during the delivery of the transfers to contract B, unaware that contract B is malicious. The attacker modifies the `fallback()` function to call `withdraw()` function, facilitating the transfer from A to B. Since the state of each variable remains unchanged throughout the whole process, the transfer condition is satisfied, leading to a continuous cycle of the “call, transfer, and callback” process until the balance in account A reaches 0 (Fatima Samreen and Alalfi, 2020).

3.2. Timestamp dependence vulnerability

In the consensus mechanism, all transactions within a block share a Timestamp to maintain contract state consistency after execution. Smart contracts may use the block Timestamp as a reference for verification or decisions based on conditional judgments. However, miners can modify the block timestamp, which may lead to errors or malicious tampering, resulting in verification or decision-making errors in smart contracts. This issue is referred to as the Timestamp dependence vulnerability (Li et al., 2023b). In Solidity, the `block.timestamp` function is commonly employed to retrieve the Timestamp of the current block and is central to the vulnerability of this issue.

In the example illustrated in Fig. 2, the contract represents a loan agreement with a repayment deadline. If the borrower fails to repay the loan by a specified deadline, penalties such as interest payments

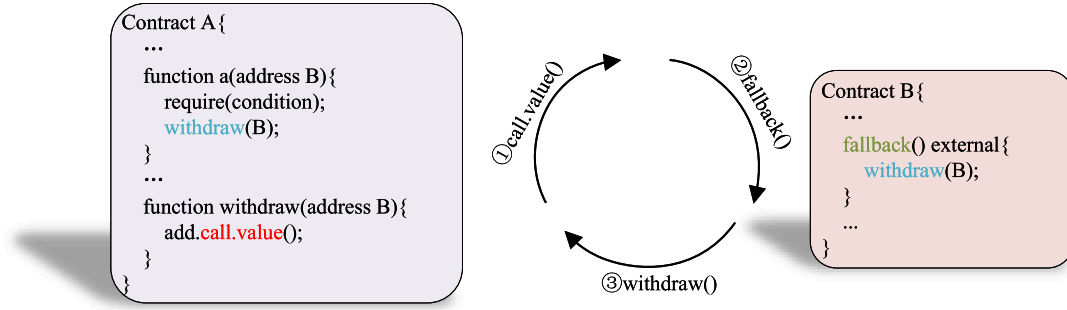


Fig. 1. Example of Reentrancy vulnerability. (a) A transfer to B using `call.value()`. (b) The transfer is completed, automatically invoking the `fallback()` function of B. (c) The attacker exploits the `withdraw()` function within `fallback()`, leading to repeated calls of `call.value()` from A to B.

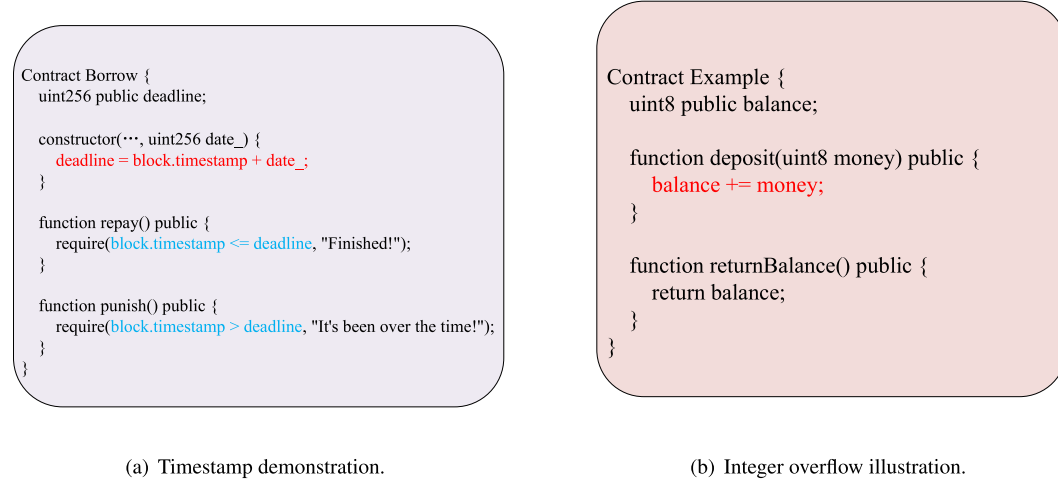


Fig. 2. Example of Timestamp and Integer overflow. (a) The `deadline` is computed based on `block.timestamp`, which also determines whether the loan is finalized based on `deadline`. (b) When the `balance` reaches the maximum value of the `uint8` type, depositing any amount causes the `balance` to reset to zero.

or mortgage confiscation apply. The deadline in the contract is determined by comparing “`block.timestamp+date_`” with “`deadline`” to assess whether the borrower has repaid within the specified time. During this process, an attacker can manipulate the mining process to alter the timestamp. It either delays the process by postponing block submission to evade the repayment punishment or advances it to impose penalties on the borrower.

3.3. Integer overflow vulnerability

Integer overflow vulnerability arises when the smart contract code requires the EVM to use unsigned integer variables with fixed limits. For instance, an 8-bit integer (`uint8`) can represent the values within `[0, 255]`. If the value exceeds the maximum value, an overflow error occurs, returning a value of 0; conversely, if the value falls below the minimum threshold, an overflow error returns the maximum value. Such vulnerabilities typically occur during numerical operations, including overflow in addition, multiplication, and subtraction.

In the code illustrated in Fig. 2, the variable `balance` in the contract represents the user’s balance, while the variable `money` denotes the amount the user wants to deposit. When the balance reaches the maximum value for the `uint8` type, any additional deposit will cause an overflow, resetting the user’s balance eventually to 0. In April 2018, the Integer overflow vulnerability in the BEC contract resulted in \$1 billion in damages, with the significant losses from the overflow causing the token’s value to plummet to zero.

4. Methods

This study presents a novel approach that combines sequence and graph models to integrate the semantic and syntactic information of smart contract codes, derived from the sequence data and structural information, respectively.

To enhance the node representations, we employ a model comprising a figure model layer and a sequence layer, effectively capturing both sequence information and structural features in the smart contract code, respectively.

In this section, we detail the overall framework and the two layers.

4.1. Model framework

The framework of Connectivity-Enhanced GCN-Transformer (CEGT) presented in this study is depicted in Fig. 3 and consists of two layers:

- (1) Figure model layer: We enhance the iteration process of GCN to improve the representation capability of the node features, effectively capturing the structural information within the contract code.
- (2) Sequence layer: We introduce a novel attention mechanism and modify the Transformer model to effectively capture the sequence information within the smart contract codes.

The complete framework transformation can be articulated as follows

$$H(x)^{l+1} = F_2(H_1(x)^{l+1} + H(x)^l) \quad (1)$$

$$H_1(x)^{l+1} = F_1(H_1(x)^l + H(x)^l) \quad (2)$$

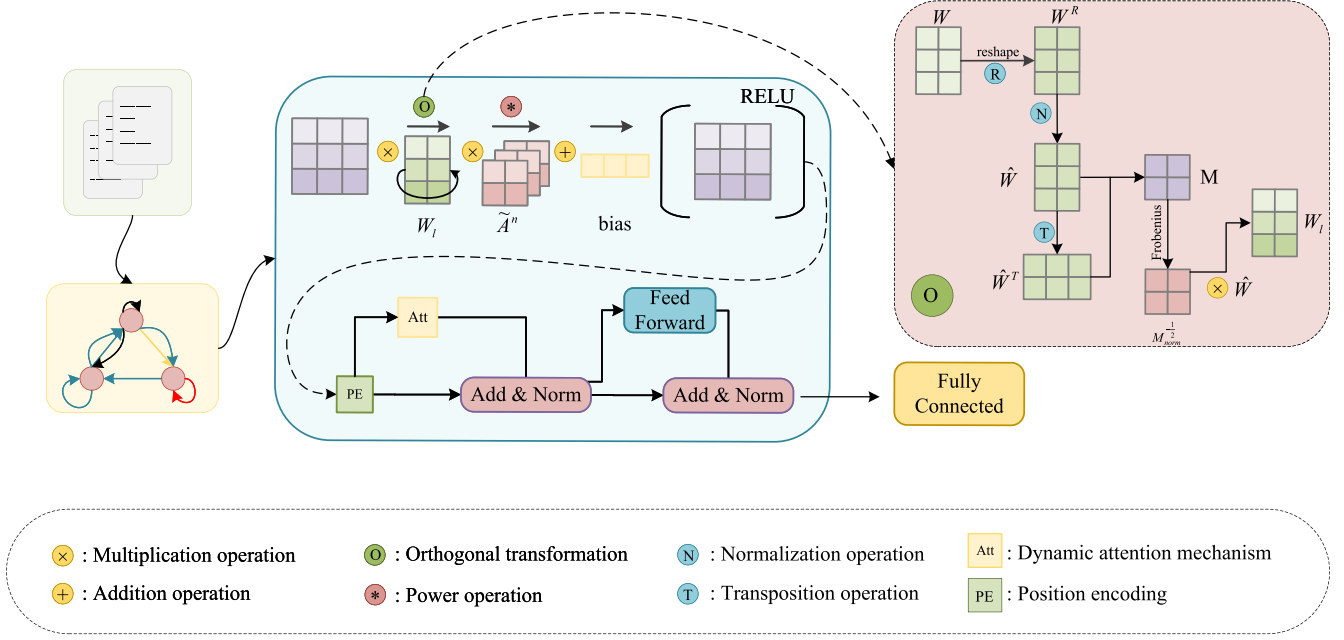


Fig. 3. CEGT architecture.

where $H(x)^l$ represents the sequence feature in layer l ; $H_1(x)^l$ denotes the structural information feature extracted through F_1 of the model; F_1 indicates the figure model layer; and F_2 signifies the sequence layer, both of which will be elaborated upon in the subsequent sections.

4.2. Figure model layer

Given the complexity and significance of interactions among nodes in the contract graph for vulnerability detection, we propose an enhanced GCN model to effectively extract the complex interaction relationships among nodes.

Algorithm 1: The pseudo-code of Orthogonal Transformation Layer (Ortho_Trans).

```

1 Initialize parameters, including  $T$  (number of iterations),
   $norm\_groups$  (number of groups for normalization), and  $eps$ 
  (small regularization constant);
2 for each input weight matrix  $weight$  do
3   Assert that  $weight.shape[0] \bmod norm\_groups = 0$ ;
4   Reshape the weight matrix into  $Z$  of shape
     ( $norm\_groups, group\_size, feature\_size$ );
5   Center each group:  $Zc = Z - \text{mean}(Z, \text{axis} = -1)$ ;
6   Compute the covariance matrix:  $S = Zc \times Zc^T$ ;
7   Add regularization:  $S = S + eps \times I$ ;
8   Normalize the covariance matrix using Frobenius norm:
      $S = \frac{S}{\|S\|_F}$ ;
9   Initialize  $B[0]$  as identity matrix of shape
     ( $feature\_size, feature\_size$ );
10  for  $t = 1$  to  $T$  do
11    Compute  $B[t + 1] = B[t] + B[t] \times (B[t]^3 \times S)$ ;
12  end
13  Apply final transformation:  $W = B[T] \times Zc / \sqrt{\|S\|_F}$ ;
14 end
15 Return the transformed weight matrix  $W$  with the same shape
   as the input weight;
```

We incorporate a residual connection and utilize the graph representation of smart contracts as input to the graph model layer, enabling

broader capture of structural information of smart contracts. In each GCN iteration, the augmented adjacency matrix $\tilde{A}$ ensures that a node's features are considered, focusing on the feature aggregation among the adjacency nodes. Therefore, to enhance the connectivity between different nodes, we propose an improved adjacency matrix $\hat{A}$, which is defined as

$$\hat{A} = \tilde{A}^n \quad (3)$$

where $\tilde{A} = A + I$; $\tilde{A}$ indicates the n th power of the enhanced adjacency matrix; the element A_{ij}^k signifies the number of different paths of length n from node i to node j .

In the GCN, each layer's iteration is defined by the following formula:

$$H(x)^{l+1} = \sigma(D^{-1/2} \tilde{A} D^{-1/2} H(x)^l W) \quad (4)$$

where $H(x) \in \mathbb{R}^{N \times d}$ represents the feature matrix; $W \in \mathbb{R}^{d \times d'}$ indicates the weight matrix that can be learnt through training; N denotes the number of nodes; d signifies the dimension of node features; and d' implies the dimension of the output feature.

In the figure model layer illustrated in this study, we adopt the orthogonal transformation introduced by Guo et al. (2022) in *Ortho-GConv* to enhance the orthogonality of W and stabilize the backward gradient. Guo et al. (2022) confirms that orthogonalizing the weights preserves both the forward embedding and the backward gradient of the signal passing through the feature transformation module. Furthermore, adding an orthogonal transformation layer can enhance the orthogonality before the initial weights are applied in the feature transformation. The main algorithms are presented in Algorithm 1. We denote the entire figure model layer as F_1 , summarizing it as follows:

$$\begin{aligned} H_1(x)^{l+1} &= F_1(H_1(x)^l + H(x)^l) \\ &= \sigma(\hat{A} \tilde{X} W_l + b_l) \end{aligned} \quad (5)$$

where W_l represents the orthogonal weight derived from W via the orthogonal transformation layer. The process for obtaining W_l is outlined as follows.

First, we normalize and mean-center the weight W to acquire $\hat{W}$:

$$W^R = \text{reshape}(W, [N_g, \frac{N}{N_g}, -1]) \quad (6)$$

$$\hat{W}_{i,j,k} = W_{i,j,k}^R - \frac{1}{d} \sum_{k=1}^d W_{i,j,k}^R \quad (7)$$

where N represents the number of nodes, N_g indicates the number of groups, and $N_g \mid N$. Meanwhile, $W^R \in \mathbb{R}^{N_g \times \frac{N}{N_g} \times d}$.

Subsequently, we employ an orthogonal projection mapping on $\hat{W}$ to derive the orthogonal weight W_l summarized as follows:

$$W_l = M_{norm}^{-1/2} \hat{W} \quad (8)$$

where M signifies $M = \hat{W} \hat{W}^T + \epsilon I$; ϵI refers to a small identity matrix added to M to ensure numerical stability; I represents the identity matrix of the same dimension as M ; ϵ indicates a very small number; and M_{norm} refers to the result obtained by $M_{norm} = \frac{M}{\|M\|_F}$ to attain unified quantization and ensure higher numerical stability. We express $\|M\|_F$ as

$$\|M\|_F = \sqrt{\sum_{i=1}^{N_g} \sum_{j=1}^{N_g} M_{ij}^2} \quad (9)$$

To mitigate the computational complexity of $M_{norm}^{-1/2}$, we employ Newton's iterative method. Within the iteration range of T , B_i converges rapidly to the square root of M_{norm} , ultimately yielding $M_{norm}^{-1/2} = B_T$.

$$B_i = 1/2(3B_{i-1} - B_{i-1}^3 M_{norm}), \quad i \in \mathbb{Z}^+ \cap [1, T] \quad (10)$$

where $B_0 = I$, and $\mathbb{Z}^+$ represents the set of positive integers.

4.3. Sequence layer

In the sequence layer, we enhance the attention mechanism derived from the Transformer model by introducing a dynamic routing attention mechanism to effectively capture the sequence information in smart contracts. In addition, we integrate a positional encoding layer, an Add&Norm layer, and a Feed-forward layer to collectively extract the sequence information of smart contracts.

Algorithm 2: The pseudo-code of Attention Layer.

```

1 Initialize parameters, including input_dim (input dimension),
  hidden_dim (hidden dimension), and num_iterations (number of
  iterations);
2 Initialize weight matrix  $W$  with shape (input_dim, hidden_dim)
  and vector  $u$  with shape (hidden_dim);
3 Initialize  $W$  with orthogonal initialization;
4 for each input tensor  $x$  with shape
  (batch_size, sequence_length, input_dim) do
5   Compute  $u_{temp} = \tanh(x \times W)$ ;
6   Compute  $u_{temp} = u_{temp} \times u$ ;
7   Compute attention scores:  $\alpha = \text{softmax}(u_{temp}, \text{axis} = 1)$ ;
8   Compute the weighted sum:  $r = \sum(\alpha \times x)$  over axis 1,
    keeping dimension;
9   for  $i = 1$  to num_iterations do
10    Compute  $\beta = \text{softmax}(x \times r^T)$  over axis 1;
11    Update  $\beta$  by expanding dimensions:  $\beta = \beta.\text{unsqueeze}(-1)$ ;
12    Update  $r = \sum(\beta \times x)$  over axis 1, keeping dimension;
13  end
14  Return the attention output  $r$  with shape
    (batch_size, 1, hidden_dim);
15 end

```

As represented in Fig. 3, we employ positional encoding to convey the positional information of the contract sequence. This method adds a fixed vector to the input embedding for each position, effectively representing both individual positions and the distances between them. The specific calculations for positional encoding are summarized as

follows:

$$\begin{cases} PE(pos, 2i) = \sin(\frac{pos}{10000^{2i/d_{model}}}) \\ PE(pos, 2i+1) = \cos(\frac{pos}{10000^{2i/d_{model}}}) \end{cases} \quad (11)$$

where pos refers to the position, i indicates the dimension index, and d_{model} signifies the size of the model's dimensionality. After incorporating the positional encoding, we obtain $\bar{X}$ as

$$\bar{X} = X + PE \quad (12)$$

In the attention layer, we propose a dynamic routing attention mechanism to emphasize sequence information within the contract. Dynamic routing is a key mechanism in capsule networks that determines how information is passed between capsules (vectorized collections of neurons). The dynamic routing attention mechanism is an approach that combines dynamic routing and attention mechanisms in capsule networks. In this mechanism, the units of the former layer adaptively assign attention weights to the latter layer units based on their relevance, thereby delivering feature information to the most appropriate units in the latter layer. This method iteratively updates the weights across multiple levels, addressing the challenge of excessive focus on the similarity matrix among input units, which can result in overfitting during the training process. The main algorithms are presented in Algorithm 2.

The dynamic routing attention mechanism generates a set of routes through learning, enabling the model to adaptively adjust the weights across different components for improved feature extraction in multi-level reasoning. Each input computes a set of "routing costs", indicating the output vector to which the current input vector should be directed. These routing costs are then forwarded to a routing selector, which uses a Softmax-based probability distribution to identify the optimal output for each input. The overall computation process is illustrated in Fig. 4.

To compute the output r_1 for the first iteration, we derive the attention distribution α from the input $\bar{X}$ and the feature matrix W , incorporating a randomly generated tensor $U \in \mathbb{R}^{d_{model} \times 1}$. Subsequently, α is employed on the input $\bar{X}$ with the output r_1 of the first iteration calculated as $\sum_{i=1}^I \alpha_i x_i$.

$$\alpha = \text{Softmax}(U \times \tanh(\bar{X}W)) \quad (13)$$

$$r_1 = \sum_{i=1}^I \alpha_i x_i \quad (14)$$

The output of each subsequent iteration is obtained by standardizing and applying weighted calculations to the output of the previous iteration, resulting in the output r_{i+1} . This process continues until the specified iteration count is reached, with the value of r obtained from the final iteration representing the final output $Att(\bar{X})$.

$$\begin{aligned} Att(\bar{X}) &= r_{i+1} \\ &= \sum_{i=1}^t \text{Softmax}(r_i \bar{X}) x_i \end{aligned} \quad (15)$$

In the Add&Norm layer and Feed-forward layers, we implement residual connections and normalization to mitigate the gradient vanishing issue. This approach enables the direct transfer of input features to subsequent layers, thereby preserving information between layers.

$$\bar{X}' = \Gamma(\bar{X} + Att(\bar{X})) \quad (16)$$

$$\Gamma(X) = \gamma \frac{X - 1/d \sum_{i=1}^d X_i}{\sqrt{1/d \sum_{j=1}^d (X_j - 1/d \sum_{k=1}^d X_k)^2 + \epsilon}} + \beta \quad (17)$$

where γ and β represent trainable parameters, and ϵ indicates a small constant to prevent dividing by zero. Finally, the output at each position is processed in the final output $F_2(x)$ through the feed-forward layer, incorporating the residual connections and normalization.

$$F(\bar{X}') = \text{RELU}(\bar{X}' W_1 + b_1) W_2 + b_2 \quad (18)$$

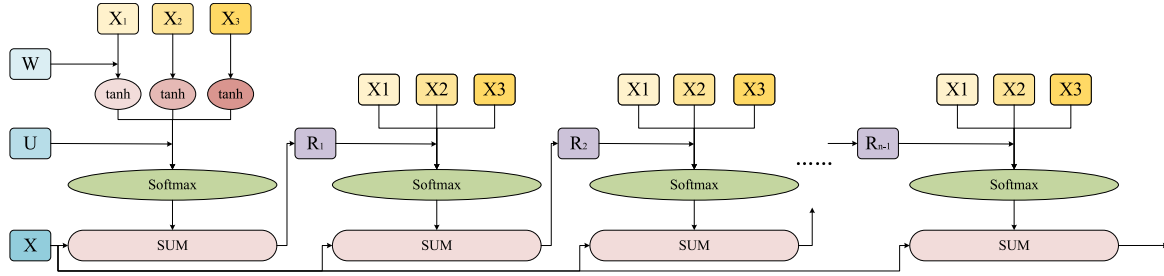


Fig. 4. The process of dynamic routing.

$$F_2(x) = \Gamma(\bar{X}') + F(\bar{X}') \quad (19)$$

where the weight matrix W_i and the bias vector b_i denote the trainable parameters used for the linear transformation; $W_1 \in \mathbb{R}^{d_{model} \times d_{ff}}$ signifies the weight matrix of the first layer linear transformation; d_{model} indicates the dimension of input/output; d_{ff} refers to the hidden layer dimension in Feed-forward; and $W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$ implies the weight matrix of the second layer linear transformation.

5. Experiments

In this study, we implemented the model framework on the DL platform Pytorch, using Python. The concrete implementation of framework CEGT has been archived permanently at Zenodo (<https://zenodo.org/records/15267620>). We collected and preprocessed multiple public datasets to address the following research questions:

- (1) RQ1: How does CEGT perform in comparison to current smart contract vulnerability detection tools?
- (2) RQ2: How does CEGT perform in comparison to other neural network-based vulnerability detection approaches?
- (3) RQ3: How do various modules influence the performance of CEGT?

Therefore, we designed three experiments to demonstrate CEGT's advancements and validate our enhancement. CEGT was compared with existing smart contract vulnerability detection tools and other ML models to confirm its superiority. Ablation experiments were also conducted to assess the effectiveness of our improvements.

We conducted our experiments using the publicly available datasets as outlined earlier, demonstrating the progressiveness of the model through the following experiments. In Experiment 1, CEGT is compared with existing smart contract vulnerability detection tools. Experiment 2 involves a comparison with nine other neural network-based model detection methods, including LSTM, RNN, GCN, BiLSTM, ReChecker, DR-GCN, TMP, CGE, and DL4SC. Experiment 3 consists of an ablation study of the model. Each experiment employs five-fold cross-validation.

5.1. Datasets and preprocessing

The datasets used in this study are sourced from publicly available, real-world smart contract datasets.

Dataset1 (Zhuang et al., 2020): This dataset is derived from the publicly available dataset used in the paper (Zhuang et al., 2020) and contains real contract code from the Ethereum. It includes 1671 Reentrancy vulnerability and 5013 Timestamp dependence vulnerability samples from smart contracts on Ethereum.

Dataset2 (Liu et al., 2021a): This dataset is sourced from GitHub's online repository (<https://github.com/Messi-Q/Smart-Contract-Dataset>) and includes several datasets covering various types of vulnerabilities, such as Reentrancy vulnerability, Timestamp dependence vulnerability, and others. For training purposes, we use 612 data samples from this dataset to detect Integer overflow vulnerability.

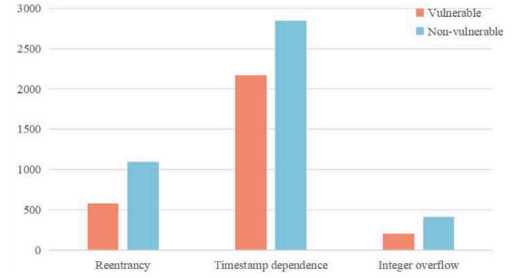


Fig. 5. Distribution of samples by category in the dataset.

Dataset3 (Qian et al., 2023): This dataset is derived from the publicly available dataset used in the paper (Qian et al., 2023), representing a small sample dataset. It consists of real contract codes from Ethereum, including 273 smart contract samples with Reentrancy vulnerabilities collected from the Ethereum blockchain.

Dataset4 (Liu et al., 2021b): This dataset originates from the publicly available dataset utilized in the paper (Liu et al., 2021b), comprising real contract codes from Ethereum. It includes 6684 smart contract samples with Reentrancy vulnerabilities collected from the Ethereum blockchain.

Dataset5 (Yashavant et al., 2022): We referenced the labels publicly available in the paper (Yashavant et al., 2022) and retrieved the contract source code from Ethereum based on the contract addresses. The samples in this dataset are all of high complexity, totaling 197 entries. We have made this dataset publicly available on GitHub for broader use.

We use Dataset1 for detecting Reentrancy vulnerability and Timestamp dependence vulnerability, and Dataset2 for detecting Integer overflow vulnerability. The specific distributions of each category are shown in Fig. 5. Dataset3, Dataset4, and Dataset5 are used for subsequent validation of CEGT's scalability across different datasets. As shown in the figure, for each vulnerability category, the proportion of vulnerabilities to non-vulnerabilities is approximately balanced, with a ratio not exceeding 1:2.

To ensure the reliability and relevance of the dataset, all data were sourced from real-world smart contracts on Etherscan, a well-known platform. During data collection, we rigorously screened and cleaned the data to exclude incomplete or invalid contract samples. Additionally, we selected a dataset that focuses on common vulnerability types in smart contracts. These include Reentrancy vulnerability, Integer overflow vulnerability, and Timestamp dependence vulnerability, which are widely recognized as critical security issues. The dataset includes a diverse range of smart contract types, ensuring its representativeness and relevance to the vulnerability detection task.

We adopt the data preprocessing method from Zhuang et al. (2020). When converting the source code of a smart contract into graph data, the nodes represent important functions, key variables, and critical operations within the contract. Depending on the type of vulnerability,

Table 1
Semantic edge classification.

Type	Semantic edge
Control – flow edges	assert{X}
	require{X}
	if{X}
	if{...} else{...}
	if{...} throw{...}
	if{...} then{...}
	if{...} revert
	while{X} do{...}
	for{X} do{...}
Data – flow edges	assign{X}
	access{X}
Forward edge	Executed sequentially
Fallback edge	Interacting with the fallback function

Table 2
Experiment environment.

Software and hardware	Configuration
CPU	Intel(R) Xeon(R) Bronze 3204 CPU @ 1.90 GHz
GPU	NVIDIA GeForce RTX 4070
Memory size	128G
Disk capacity	4.5T
CUDA	12.4
Python	3.10
Pytorch	2.5.0

Table 3
Hyperparameter setting.

Hyperparameter	Value
lr	[0.01, 0.03 0.003]
lr_decay_steps	1,3
dropout	[0.03, 0.05]
filters	64,64,64
n_hidden	[64, 128, 256]
epoch	50
batch_size	[32, 64]
multi_head	4

the focus of these functions varies: for Reentrancy vulnerability, the primary concern is the **call.value()** function; for Timestamp dependence vulnerability, the focus is on **block.timestamp**; and for Integer overflow vulnerability, the key operations involve add, sub, mul, and other arithmetic functions. Additionally, the importance of the **fallback()** function is highlighted in the context of the vulnerability mechanism. Directed edges in the graph are used to capture semantic dependencies between nodes, categorized into four types: control flow, data flow, forward flow, and fallback flow. The specific semantic edge categories are summarized in Table 1.

5.2. Experimental setup

All experiments are conducted on a workstation equipped with an NVIDIA GeForce RTX 4070 GPU and an Intel(R) Xeon(R) Bronze 3204 CPU @ 1.90 GHz. The specific hardware and software settings are presented in Table 2.

The goal of setting the hyperparameters is to maximize the accuracy of vulnerability detection. We set the word embedding dimension to 512, the number of epochs to 50, the optimizer to Adam, and use the cross-entropy function as the loss function. Additionally, we apply five-fold cross-validation in our experiments. The range of settings for the remaining hyperparameters is shown in Table 3.

5.3. Evaluation metrics

This study employs four performance comparison indicators: accuracy (ACC), recall (Recall), precision (Precision), and F1-score (F1),

Table 4
Confusion matrix representation.

	Predicate		
	Positive	Negative	
True	Positive	TP	FN
	Negative	FP	FN

which are computed using the formula provided. These metrics are derived from the confusion matrix presented in Table 4:

$$Acc = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Recall = \frac{TP}{TP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$F1 = \frac{2Recall * Precision}{Recall + Precision}$$

where TP represents the count of categories with both positive predicted and true label values; TN implies the number of categories with negative predicted values and false label values; FN indicates the number of categories with negative predicted values but positive true labels; and FP refers to the number of categories with positive predicted values and negative labels.

5.4. Comparison with existing vulnerability detection tools (RQ1)

In RQ1, we evaluate CEGT against current smart contract vulnerability detection tools to validate the advancements of CEGT. Due to issues with version-matching smart contracts and the discontinuation of certain tools such as SmartCheck, we compare CEGT with Oyente (Luu et al., 2016), Mythril (Consensys, 2017), and Slither (Feist et al., 2019) tools in detecting Reentrancy, Timestamp dependence, and Integer overflow vulnerabilities. Oyente (Luu et al., 2016) is a vulnerability detection tool that utilizes symbolic execution, taking smart contract bytecode as input and exploring accessible paths through CFG. Slither (Feist et al., 2019) is a static analysis framework that visualizes contract details via various vulnerability detectors. Mythril (Consensys, 2017) is a security analysis tool for EVM bytecode, employing symbolic execution, SMT solving, and stain analysis to identify security vulnerabilities in smart contracts built across multiple EVM-compatible blockchains. We compared four indicators: ACC, Recall, Precision, and F1, with specific comparison results presented in Table 5.

In detecting Reentrancy vulnerabilities, Slither is the top-performing tool, achieving 87.91% ACC and 79.75% F1. CEGT improves upon Slither by 7.51% in ACC, 6.54% in Recall, 18.87% in Precision, and 13.72% in F1. For Timestamp dependence vulnerability detection, Slither again performs best. While Oyente exhibits the highest ACC, its lower Recall results in a reduced F1. CEGT enhances ACC by 44.38%, Recall by 36.43%, Precision by 28.84%, and F1 by 42.29% compared with Slither. In detecting Integer overflow vulnerability, we only compare CEGT with Oyente and Slither, as Mythril currently does not support this detection. Among them, Oyente demonstrates the best performance in Integer overflow vulnerability detection. However, CEGT shows improvements of 50.84% in ACC, 26.26% in Recall, 47.84% in Precision, and 37.42% in F1 compared with Oyente. These results indicate that CEGT exhibits superior detection performance relative to existing smart contract vulnerability detection tools. Our analysis of traditional detection tools' source code reveals that these tools strictly enforce fixed expert-defined rules to detect vulnerabilities, leading to their poor performance.

As illustrated in Fig. 6, we visualize the vulnerability detection results of CEGT in comparison to traditional vulnerability detection tools. Specifically, the graph illustrates the comparison results for Reentrancy,

Table 5

Comparison with existing detection tools.

Method	Reentrancy (%)				Timestamp (%)				Integeroverflow (%)			
	ACC	Recall	Precision	F1	ACC	Recall	Precision	F1	ACC	Recall	Precision	F1
Oyente (Luu et al., 2016)	66.30	79.45	42.96	55.77	59.46	19.44	45.16	27.18	42.91	62.22	47.46	53.85
Slither (Feist et al., 2019)	87.91	89.04	72.22	79.75	46.42	58.66	48.17	52.90	35.27	47.78	24.71	32.58
Mythril (Consensys, 2017)	63.50	72.97	40.30	51.92	43.84	38.55	44.52	41.32	/	/	/	/
CEGT	95.42	95.58	91.09	93.47	90.80	87.50	90.46	89.33	93.75	88.48	95.30	91.27

As Mythril does not support the detection of Integer overflow vulnerabilities, this type of detection is only compared with Oyente and Slither.

Table 6

Contrast results with other models.

Method	Reentrancy (%)				Timestamp (%)				Integeroverflow (%)			
	ACC	Recall	Precision	F1	ACC	Recall	Precision	F1	ACC	Recall	Precision	F1
LSTM (Hochreiter and Schmidhuber, 1997)	76.19	72.38	78.35	75.25	81.67	82.17	66.15	73.30	51.75	60.53	36.50	45.54
RNN	65.50	46.67	37.40	51.53	74.85	73.89	56.86	64.27	54.39	50.00	36.54	42.22
GCN	81.25	66.25	76.43	70.97	85.27	86.77	79.85	82.69	62.90	52.38	45.83	48.89
BILSTM	77.14	85.71	82.20	83.26	82.73	78.39	79.74	79.06	61.40	60.53	44.23	51.11
ReChecker (Qian et al., 2020)	78.57	70.48	84.10	76.68	82.47	62.29	76.27	68.53	61.40	68.42	44.83	54.17
DR-GCN (Zhuang et al., 2020)	84.50	75.51	63.58	68.41	83.63	69.92	71.99	70.46	83.16	71.21	76.47	73.55
TMP (Zhuang et al., 2020)	88.74	88.58	80.51	84.35	86.87	83.50	84.83	84.05	87.91	74.51	87.36	80.42
CGE (Liu et al., 2021b)	91.59	84.29	90.28	87.19	88.12	85.02	86.06	85.54	/	/	/	/
DL4SC (Liu et al., 2024a)	92.92	94.34	87.01	90.03	89.46	87.30	89.86	88.28	91.11	86.43	88.22	86.92
CEGT	95.42	95.58	91.09	93.47	90.80	87.50	90.46	89.33	93.75	88.48	95.30	91.27

The right side of the table presents the detection comparison results for Integer overflow vulnerabilities, which are not supported by CEGT.

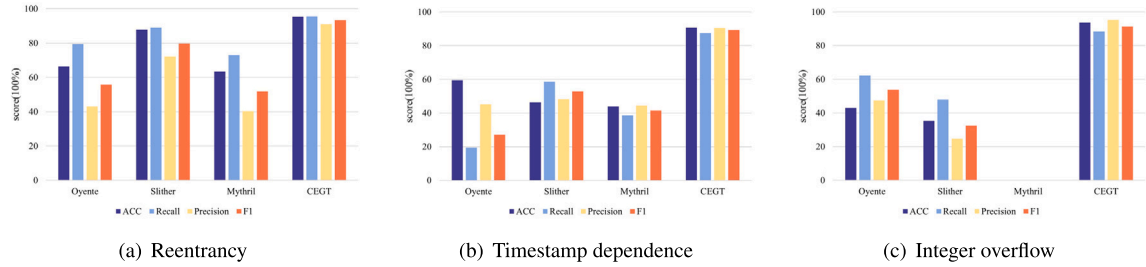


Fig. 6. Comparison with current detection tools. (a) Detection and comparison result for the Reentrancy vulnerability. (b) Contrast in detection result for the Timestamp dependence vulnerability. (c) Outcome of detection comparison for the Integer overflow vulnerability. Since Mythril does not support detecting the Integer overflow vulnerability, its comparison is limited to Oyente and Slither.

Timestamp dependence, and Integer overflow vulnerability detection. The tools, from left to right are Oyente, Slither, Mythril, and CEGT. The figure clearly shows that CEGT significantly outperforms the traditional vulnerability detection tools.

5.5. Comparison with ML methods (RQ2)

In RQ2, we compare CEGT with nine neural network-based detection methods, namely LSTM, RNN, GCN, BiLSTM, ReChecker (Qian et al., 2020), DR-GCN (Zhuang et al., 2020), TMP (Zhuang et al., 2020), CGE (Liu et al., 2021b), and DL4SC (Liu et al., 2024a). This comparison focuses on detecting Reentrancy, Timestamp dependence, and Integer overflow vulnerabilities. The detection performance of different models is evaluated using four metrics: ACC, Recall, Precision, and F1, with the detailed comparison results presented in Table 6.

As shown in Table 6, DL4SC achieves the highest performance among the three categories of vulnerability detection. CEGT enhances the ACC, Recall, Precision, and F1 by 2.5%, 1.24%, 4.048%, and 3.44%, respectively, compared with DL4SC. In Timestamp dependence vulnerability detection, CEGT outperforms DL4SC with improvements of 1.34%, 0.2%, 0.6/%, and 1.05% in ACC, Recall, Precision, and F1, respectively. For Integer overflow vulnerability detection, since CGE does not support the detection of this vulnerability type, comparisons are made only with other models. CEGT shows significant enhancements over DL4SC, with increases of 2.64% in ACC, 2.05% in Recall,

7.08% in Precision, and 4.35% in F1. Among the DL-based models, GCN outperforms LSTM, RNN, and BiLSTM, as these models fail to leverage structural information — such as calling relationships — and treat smart contracts merely as text, disregarding syntactic structure. This one-dimensional approach limits their detection performance. By contrast, CEGT excels in all detection scenarios by expanding the node features in the graph representation from local to global, effectively integrating both local and global feature information through the combination of sequence models.

As shown in Fig. 7, we illustrate a comparison of CEGT's vulnerability detection performance against other neural network and DL-based methods. In particular, the graph highlights a comparative analysis of Reentrancy, Timestamp dependence, and Integer overflow vulnerability detection results. From left to right, the methods shown are LSTM, RNN, GCN, BiLSTM, ReChecker, DR-GCN, TMP, CGE, DL4SC, and CEGT. The graph demonstrates that CEGT outperforms the other neural networks and DL-based methods.

5.6. Ablation experiment (RQ3)

CEGT is based on GCN, with improvements categorized into three components. The matrix improvement is denoted as M , the orthogonal transformation of the weight matrix is implied as O , and the attention mechanism in the sequence model is indicated as A . To evaluate the

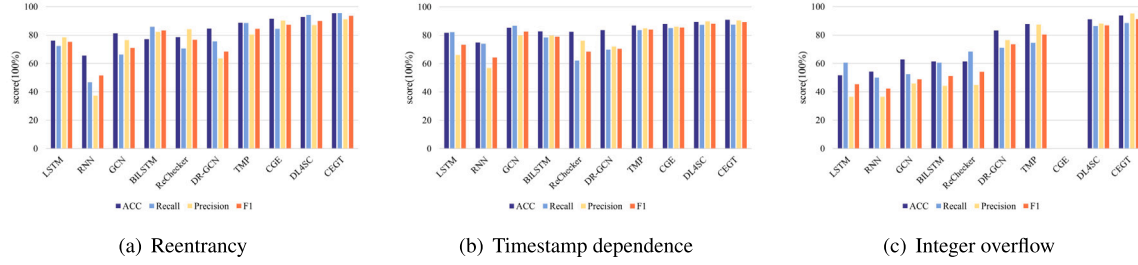


Fig. 7. Comparison of results with other models. (a) Detection and comparison results for the Reentrancy vulnerability. (b) Contrast in detection results for the Timestamp dependency vulnerability. (c) Detection and comparison results for the Integer overflow vulnerability, which is not supported by CGE.

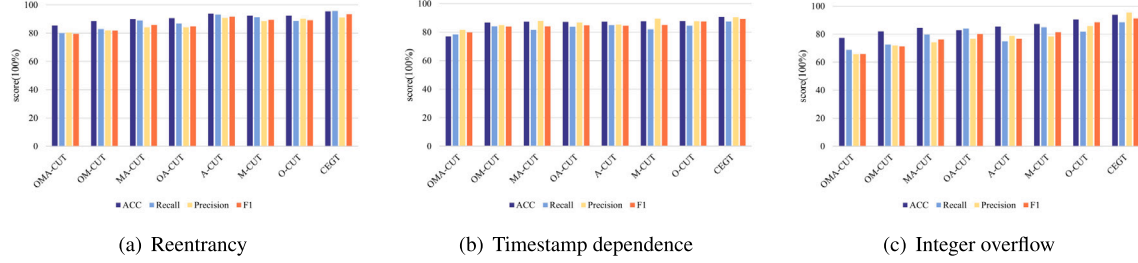


Fig. 8. Outcomes of the ablation experiments. (a) Detection and comparison outcome for the Reentrancy vulnerability. (b) Contrast in detection outcome for the Timestamp dependency vulnerability. (c) Outcome of detection comparison for the Integer overflow vulnerability.

effectiveness of these enhancements within our fusion model, we systematically remove each component and compare the resulting variants with the original model. The modified models are designated as *OMA-CUT*, *OM-CUT*, *MA-CUT*, *OA-CUT*, *A-CUT*, *M-CUT*, and *O-CUT*. The detection performance of various model variants is evaluated using four metrics: ACC, Recall, Precision, and F1. The results are presented in Table 7 and Fig. 8.

The results demonstrate that CEGT surpasses its variants in detecting all three vulnerability types, highlighting the necessity of integrating graph and sequence models along with their enhancements. Furthermore, for Reentrancy and Timestamp dependence vulnerability detection, the orthogonal transformation alone yields better performance than either adjacency matrix exponentiation or attention mechanism individually. For Integer overflow vulnerability detection, retaining the adjacency matrix exponentiation operation is more effective than maintaining either of the other two components independently. Since CEGT primarily relies on node input, enhancements based solely on the sequence model do not optimize performance. Conversely, improving the graph model, particularly by refining the iteration process of the basic GCN, significantly enhances the model's capacity to capture global features and improve detection performance. Meanwhile, experimental results show that our dynamic routing attention mechanism adjusts attention weights based on the complex dependencies between nodes and edges in the graph. By simulating a dynamic routing process, it continuously adapts its attention paths. This allows it to focus more precisely on critical code segments with potential vulnerabilities. Ordinary attention mechanisms struggle to handle this task effectively.

6. Discussion

Vulnerabilities in smart contracts can result in significant financial losses, execution failures, and market instability, which may severely damage a project's reputation, erode investor trust, and potentially lead to legal disputes and increased regulatory scrutiny. These issues collectively hinder the broader adoption of blockchain technology, underscoring the critical importance of effective smart contract vulnerability detection.

To evaluate the effectiveness of the proposed model, we compared the performance of our CEGT model against traditional vulnerability

detection tools, such as Mythril and Oyente, using publicly available smart contract datasets that contain real-world code with associated vulnerabilities. We design three experiments. Experiments I and II reveal that CEGT achieves optimal detection performance by effectively integrating local and global feature information. Furthermore, the ablation study in Experiment III confirms that each module within the model contributes to its enhanced performance. Unlike traditional tools, which rely on rigid, rule-based approaches, CEGT model comprehensively captures vulnerability patterns by extending node features and incorporating sequence models.

Furthermore, we also validated the scalability of CEGT on larger datasets through additional experiments. By training on a larger dataset for Reentrancy vulnerability, Dataset4, we achieved an F1 score of 93.39%. Dataset4 contains 6684 samples, approximately four times the size of the dataset we previously used. The overall training time correspondingly increased from 1 h to 3.5 h. Additionally, we utilized a small sample dataset, Dataset3, which consists of only 273 entries and required merely 0.5 h of training time. Notably, the memory usage of CEGT during training remained relatively consistent across datasets of varying sizes. Therefore, the training time of CEGT is somewhat related to the size of the dataset. As the sample data increases, effectively parallelizing and improving training efficiency when using larger-scale data becomes crucial. This will also be a key direction for our future focus and research.

Additionally, we collected 197 highly complex smart contracts from Ethereum to form Dataset5. The average size of the smart contracts in Dataset5 is 14,625B, which is 48 times the average size of the original dataset. The average number of lines of code, excluding comments, is 372, which is 28 times the average line count of the original dataset. Moreover, the average number of functions in Dataset5 is nearly 30 times that of the original dataset. We tested these 197 entries using the pre-trained Reentrancy vulnerability detection model, achieving an F1 score of 89.00%, which is a 4.47% decrease compared to the original dataset. This is because CEGT's detection is based on nodes and edges, and a single file may contain multiple contracts with nested calling relationships between them. Consequently, for highly complex smart contracts, the number of nodes and edges in their contract graphs is significantly higher than that of short-text code. This also results in CEGT requiring more time and further improvement in accuracy when

Table 7
Results of the ablation comparison experiment.

Method	Reentrancy (%)				Timestamp (%)				Integeroverflow (%)			
	ACC	Recall	Precision	F1	ACC	Recall	Precision	F1	ACC	Recall	Precision	F1
OMA – CUT	85.35	80.13	80.35	79.57	76.88	78.41	81.73	79.71	77.29	68.69	65.71	65.87
OM – CUT	88.61	82.98	81.93	81.82	86.83	84.02	84.85	83.86	82.11	72.71	71.81	71.37
MA – CUT	90.05	88.99	84.16	85.92	87.32	81.72	87.90	84.12	84.61	79.62	74.25	76.14
OA – CUT	90.69	86.79	84.27	84.84	87.04	83.70	86.77	84.77	82.93	83.92	76.92	80.13
A – CUT	93.94	93.13	90.89	91.80	87.29	84.81	85.31	84.46	85.52	75.00	78.89	76.84
M – CUT	92.45	91.24	88.60	89.38	87.61	82.07	89.46	85.06	87.29	85.00	78.33	81.36
O – CUT	92.37	88.67	90.34	89.27	87.70	84.43	87.57	87.46	90.42	81.82	85.71	88.56
CEGT	95.42	95.58	91.09	93.47	90.80	87.50	90.46	89.33	93.75	88.48	95.30	91.27

M denotes the enhancement of the adjacency matrix, O represents the orthogonal transformation of the weight matrix, and A signifies the attention mechanism within the sequence model.

Example1	Example2
<pre>function distributeBTR(address[] addresses) onlyOwner { for (uint i = 0; i < addresses.length; i++) { balances[owner] -= 2000 * 10**8; balances[addresses[i]] += 2000 * 10**8; Transfer(owner, addresses[i], 2000 * 10**8); } }</pre>	<pre>contract PrivateBank { uint car; function add(uint8 balance, uint8 store) { uint8 sum = balance + store; car = sum; } }</pre>

Fig. 9. In other cases of Integer overflow vulnerability detection failure. Example1 illustrates a situation where the operation on a constant is not detected, leading to a detection failure. Example2 shows that inaccurate detection due to overflow in non-uint256 data types results in a failure to detect the vulnerability.

detecting such high-complexity code. In the future, we will further investigate vulnerability detection for highly complex smart contracts to enhance the model's scalability.

In real-world cases, CEGT successfully detects vulnerabilities missed by traditional tools. As shown in Fig. 9, in a contract with a potential Integer overflow, CEGT identifies the vulnerability by analyzing function call sequences and state changes, while Mythril fails due to its inability to handle constant arithmetic and non-uint256 data types. This highlights CEGT's superior capability in capturing complex patterns through its integration of local and global features.

However, it is important to note that while our model demonstrates high accuracy in detecting the three studied vulnerability types, the smart contracts in the datasets we used are mostly short-form code, and the model only detects for three vulnerability types. Future work will optimize the model structure, increase the diversity of training data, improve the adaptability to complex contracts, and implement multi-vulnerability detection to address a wider range of smart contract security issues.

7. Threats to validity

While CEGT performs well on existing datasets, we also acknowledge the challenges currently faced. The publicly available dataset we used focuses only on three types of smart contract vulnerabilities, which may impact the model's generalization ability. In the future, we will detect more types of vulnerabilities and realize multi-label vulnerability detection. The attack patterns of smart contract vulnerabilities evolve over time, making it impossible to fully address future emerging attack modes. Considering the powerful reasoning capabilities of large language models, we will also explore their use for effective smart contract vulnerability detection in the future. We also recognize the challenge posed by adversarial samples. Future research will focus on enhancing CEGT's robustness to counter malicious contract evasion strategies.

8. Conclusion

This study presents a model for detecting vulnerabilities in smart contracts. The model accounts for both contextual relationships and node features within the source code while emphasizing semantic

logical relationships through intermingled connectivity. In addition, the study enhances node connectivity by refining the graph model to ensure stability and demonstrate the efficacy of these advancements through experiments. Furthermore, the study introduces a dynamic routing attention mechanism that significantly enhances model accuracy compared with traditional attention mechanisms, as shown by extensive experimental results. Comprehensive experiments indicate that our proposed model exhibits notable improvements. Our work currently focuses on single vulnerability detection for three types of vulnerabilities. A future objective of our study will be to enhance the experimental data and model structure to enable multi-vulnerability detection in smart contracts.

CRedit authorship contribution statement

Jiandong Shang: Conceptualization. **Jiaru Li:** Writing – original draft, Resources, Data curation. **Yizhe Sui:** Validation. **Hengliang Guo:** Conceptualization. **Xu Gao:** Resources, Conceptualization. **Du-juan Zhang:** Supervision. **Yang Guo:** Supervision. **Gang Wu:** Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Ashizawa, N., Yanai, N., Cruz, J.P., Okamura, S., 2022. Eth2Vec: Learning contract-wide code representations for vulnerability detection on Ethereum smart contracts. *Blockchain: Res. Appl.* 3, 100101. <http://dx.doi.org/10.1016/j.bcr.2022.100101>.
- Cao, Yuanlong, Jiang, Fan, Xiao, Jianmao, et al., 2023. A smart contract vulnerability detection method and system: CN202211410823.2 [P] CN115828258A[2023-03-21].(in Chinese).

- Casino, F., Dasaklis, T.K., Patsakis, C., 2019. A systematic literature review of blockchain-based applications: Current status, classification and open issues. *Telemat. Inform.* 36, 55–81. <http://dx.doi.org/10.1016/j.tele.2018.11.006>.
- Consensus, 2017. GitHub - Consensus/mythril: Security analysis tool for EVM bytecode. Supports smart contracts built for ethereum, heder, quorum, vechain, rootstock, tron and other EVM-compatible blockchains. GitHub. (n.d.). <https://github.com/Consensus/mythril>.
- Fatima Samreen, N., Alalfi, M.H., 2020. Reentrancy vulnerability identification in ethereum smart contracts. In: 2020 IEEE International Workshop on Blockchain Oriented Software Engineering. IWBOSE, IEEE, <http://dx.doi.org/10.1109/iwbose50093.2020.9050260>.
- Feist, J., Grieco, G., Groce, A., 2019. Slither: A static analysis framework for smart contracts. In: 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain. WETSEB, IEEE, <http://dx.doi.org/10.1109/wetseb.2019.00008>.
- Gong, K., Song, X., Wang, N., Wang, C., Zhu, H., 2023. SCGformer: Smart contract vulnerability detection based on control flow graph and transformer. *IET Blockchain* 3, 213–221. <http://dx.doi.org/10.1049/bic2.12046>.
- Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S.K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., Zhou, M., 2020. GraphCodeBERT: Pre-training code representations with data flow. *arXiv.org* <https://arxiv.org/abs/2009.08366>.
- Guo, K., Zhou, K., Hu, X., Li, Y., Chang, Y., Wang, X., 2022. Orthogonal graph neural networks. *Proc. AAAI Conf. Artif. Intell.* 36, 3996–4004. <http://dx.doi.org/10.1609/aaai.v36i4.20316>.
- Gupta, R., Patel, M.M., Shukla, A., Tanwar, S., 2022. Deep learning-based malicious smart contract detection scheme for internet of things environment. *Comput. Electr. Eng.* 97, 107583. <http://dx.doi.org/10.1016/j.compeleceng.2021.107583>.
- He, L., Zhao, X., Wang, Y., 2024. ParSE: Efficient detection of smart contract vulnerabilities via parallel and simplified symbolic execution. In: Proceedings of the 2024 IEEE/ACM 46th Int'l Conf. on Software Eng.: Companion Proceedings. ACM, New York, NY, USA, pp. 272–273. <http://dx.doi.org/10.1145/3639478.3643066>.
- Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. *Neural Comput.* 9, 1735–1780. <http://dx.doi.org/10.1162/neco.1997.9.8.1735>.
- Hu, T., Liu, X., Chen, T., Zhang, X., Huang, X., Niu, W., Lu, J., Zhou, K., Liu, Y., 2021. Transaction-based classification and detection approach for Ethereum smart contract. *Inf. Process. Manage.* 58, 102462. <http://dx.doi.org/10.1016/j.ipm.2020.102462>.
- Huang, T.H.-D., 2018. Hunting the ethereum smart contract: Color-inspired inspection of potential attacks. *arXiv.org* <https://arxiv.org/abs/1807.01868>.
- Jeon, S., Lee, G., Kim, H., Woo, S.S., 2023. Design and evaluation of highly accurate smart contract code vulnerability detection framework. *Data Min. Knowl. Discov.* 38, 888–912. <http://dx.doi.org/10.1007/s10618-023-00981-1>.
- Li, H., Dang, R., Yao, Y., Wang, H., 2023a. A review of approaches for detecting vulnerabilities in smart contracts within web 3.0 applications. *Blockchains* 1, 3–18. <http://dx.doi.org/10.3390/blockchains1010002>.
- Li, J., Lu, G., Gao, Y., Gao, F., 2023b. A smart contract vulnerability detection method based on multimodal feature fusion and deep learning. *Math.* 11, 4823. <http://dx.doi.org/10.3390/math11234823>.
- Liao, J.-W., Tsai, T.-T., He, C.-K., Tien, C.-W., 2019. SoliAudit: Smart contract vulnerability assessment based on machine learning and fuzz testing. In: 2019 Sixth Int'l Conf. on Internet of Things: Systems, Management and Security. IOTSMS, IEEE, <http://dx.doi.org/10.1109/iotsms48152.2019.8939256>.
- Liu, J., Liang, C., Feng, J., Xiao, A., Zeng, H., Wu, Q., Yu, T., 2023. A multi-feature fusion-based automatic detection method for high-severity defects. *Electron.* 12, 3075. <http://dx.doi.org/10.3390/electronics12143075>.
- Liu, H., Liu, C., Zhao, W., Jiang, Y., Sun, J., 2018. S-gram: towards semantic-aware security auditing for Ethereum smart contracts. In: Proc. of the 33rd IEEE/ACM Int'l Conf. on Automated Soft. Eng.. ASE, ACM, New York, NY, USA, <http://dx.doi.org/10.1145/3238147.3240728>.
- Liu, Z., Qian, P., Wang, X., Zhu, L., He, Q., Ji, S., 2021a. Smart contract vulnerability detection: From pure neural network to interpretable graph feature and expert pattern fusion. In: Proc. of the Thirtieth Int'l Jt. Conf. on Artificial Intelligence, Int'l Jt. Conf. on Artificial Intelligence Organization. California, pp. 2751–2759. <http://dx.doi.org/10.24963/ijcai.2021/379>.
- Liu, Z., Qian, P., Wang, X., Zhuang, Y., Qiu, L., Wang, X., 2021b. Combining graph neural networks with expert knowledge for smart contract vulnerability detection. *IEEE Trans. Knowl. Data Eng.* <http://dx.doi.org/10.1109/tkde.2021.3095196>, 1–1.
- Liu, B., Shi, L., Cai, Z., Li, M., 2012. Software vulnerability discovery techniques: A survey. In: 2012 Fourth Int'l Conf. on Multimedia Information Networking and Security. IEEE, pp. 152–156. <http://dx.doi.org/10.1109/mines.2012.202>.
- Liu, Y., Wang, C., Ma, Y., 2024a. DL4SC: a novel deep learning-based vulnerability detection framework for smart contracts. *Autom. Soft. Eng.* 31, <http://dx.doi.org/10.1007/s10515-024-00418-z>.
- Liu, R., Wang, Y., Xu, H., Liu, B., Sun, J., Guo, Z., Ma, W., 2024b. Source code vulnerability detection: Combining code language models and code property graphs. *arXiv.org* <https://arxiv.org/abs/2404.14719>.
- Luu, L., Chu, D.-H., Olickel, H., Saxena, P., Hobor, A., 2016. Making smart contracts smarter. In: Proc. of the 2016 ACM SIGSAC Conf. on Comput. and Communications Security. ACM, New York, NY, USA, pp. 254–269. <http://dx.doi.org/10.1145/2976749.2978309>.
- Mehar, M.I., Shier, C.L., Giambattista, A., Gong, E., Fletcher, G., Sanayhie, R., Kim, H.M., Laskowski, M., 2019. Understanding a revolutionary and flawed grand experiment in blockchain. *J. Cases Inf. Technol.* 21, 19–32. <http://dx.doi.org/10.4018/jcit.2019010102>.
- Qian, P., Liu, Z., He, Q., Zimmermann, R., Wang, X., 2020. Towards automated reentrancy detection for smart contracts based on sequential models. *IEEE Access* 8, 19685–19695. <http://dx.doi.org/10.1109/access.2020.2969429>.
- Qian, P., Liu, Z., Yin, Y., He, Q., 2023. Cross-modality mutual learning for enhancing smart contract vulnerability detection on bytecode. In: Proc. of the ACM Web Conf. 2023. ACM, New York, NY, USA, pp. 2220–2229. <http://dx.doi.org/10.1145/3543507.3583367>.
- Tann, W.J.-W., Han, X.J., Gupta, S.S., Ong, Y.-S., 2018. Towards safer smart contracts: A sequence learning approach to detecting security threats. *arXiv.org* <https://arxiv.org/abs/1811.06632>.
- Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., Alexandrov, Y., 2018. SmartCheck. In: Proc. of the 1st Int'l Workshop on Emerging Trends in Soft. Eng. for Blockchain. ACM, New York, NY, USA, <http://dx.doi.org/10.1145/3194113.3194115>.
- Wang, L., Cheng, H., Zheng, Z., Yang, A., Zhu, X., 2021a. Ponzi scheme detection via oversampling-based long short-term memory for smart contracts. *Knowl.-Based Syst.* 228, 107312. <http://dx.doi.org/10.1016/j.knosys.2021.107312>.
- Wang, Y., Jia, P., Liu, L., Huang, C., Liu, Z., 2020. A systematic review of fuzzing based on machine learning techniques. *Plos One* 15, e0237749. <http://dx.doi.org/10.1371/journal.pone.0237749>.
- Wang, W., Song, J., Xu, G., Li, Y., Wang, H., Su, C., 2021b. ContractWard: Automated vulnerability detection models for ethereum smart contracts. *IEEE Trans. Netw. Sci. Eng.* 8, 1133–1144. <http://dx.doi.org/10.1109/tNSE.2020.2968505>.
- Wang, Yifeng, Zeng, Cheng, Quan, Qingyu, et al., 2024. A defect detection method for smart contracts based on multi feature fusion [J/OL]. *Comput. Eng.* 50, 1–10. <http://kns.cnki.net/kcms/detail/31.1289.tp.20231229.1442.012.html>.
- Wu, H., Peng, Y., He, Y., Fan, J., 2024. A review of deep learning-based vulnerability detection tools for ethernet smart contracts. *Comput. Model. Eng. Sci.* 140, 77–108. <http://dx.doi.org/10.32604/cmes.2024.046758>.
- Wu, H., Zhang, Z., Wang, S., Lei, Y., Lin, B., Qin, Y., Zhang, H., Mao, X., 2021. Peculiar: Smart contract vulnerability detection based on crucial data flow graph and pre-training techniques. In: 2021 IEEE 32nd Int'l Symposium on Software Reliability Engineering. ISSRE, IEEE, <http://dx.doi.org/10.1109/issre52982.2021.00047>.
- Yashavant, Chavhan Sujeet, et al., 2022. ScrawlD: A dataset of real world ethereum smart contracts labelled with vulnerabilities. *arXiv* [abs/2202.11409](https://arxiv.org/abs/2202.11409), n. pag.
- You, Juan, Xia, Song, Li, Junquan, 2012. A survey on formal methods using in software development. In: IET Int'l Conf. on Information Science and Control Eng. 2012. ICISCE 2012, Institution of Eng. and Technology, <http://dx.doi.org/10.1049/cp.2012.2353>, 2.40-2.40.
- Zhang, Z., Lei, Y., Yan, M., Yu, Y., Chen, J., Wang, S., Mao, X., 2022. Reentrancy vulnerability detection and localization: A deep learning based two-phase approach. In: Proc. of the 37th IEEE/ACM Int'l Conf. on Automated Software Engineering. ACM, New York, NY, USA, <http://dx.doi.org/10.1145/3551349.3560428>.
- Zhen, Z., Zhao, X., Zhang, J., Wang, Y., Chen, H., 2024. DA-GNN: A smart contract vulnerability detection method based on dual attention graph neural network. *Comput. Netw.* 242, 110238. <http://dx.doi.org/10.1016/j.comnet.2024.110238>.
- Zhou, K., Cheng, J., Li, H., Yuan, Y., Liu, L., Li, X., 2021. SC-VDM: A lightweight smart contract vulnerability detection model. In: Communications in Comput. and Information Science. Springer Singapore, Singapore, pp. 138–149. http://dx.doi.org/10.1007/978-981-16-7476-1_13.
- Zhuang, Y., Liu, Z., Qian, P., Liu, Q., Wang, X., He, Q., 2020. Smart contract vulnerability detection using graph neural network. In: Proc. of the Twenty-Ninth Int'l Jt. Conf. on Artificial Intelligence, Int'l Jt. Conf. on Artificial Intelligence Organization. California, pp. 3283–3290. <http://dx.doi.org/10.24963/ijcai.2020/454>.